

Ready to start working?

This is a preview of the project. Enter work mode to work on your task.

Enter Work Mode

Qualification Overview

Welcome to this new qualification! Seeing this qualification means that you have been doing well on a variety of coding projects on the platform. We're therefore giving you the opportunity to complete and excel in this qual to gain access to a series of long-running, fun coding projects with file uploads and agentic tool uses. This qualification consists of three sections:

- **Section 1 - Survey:** Report your terminal agent / coding agent tool use experience.
- **Section 2 - Coding Agent Setup:** Follow the instructions to set up Codex, Gemini, or Claude Code and a terminal logging tool.
- **Section 3 - Talk with Coding Agent:** Follow the instructions to locate a suitable codebase to converse with the model about, submit a conversation with a terminal coding agent tool, and provide some light evals data.

⚠ You're given 2 chances for this qualification, and as long as you did well in 1 of them, you will get access to a series of fun, long-running, higher-paid (often \$60/hr and above) coding projects with file uploads and agentic tool uses, which could include but not be limited to the following project types. ⚠

- Having a conversation with a terminal agent in an **external AI interface** about topics similar to the mini-tasks you will complete in this qual.
- Review and rewrite LLM-written code diff reviews about PRs of open-source GH repos.
- Having and evaluating conversations with different terminal agents using frontier models.
- *[Rarer cases]* Contribute to building advanced, challenging agentic coding datasets that crash frontier models.

Note that you do **NOT** need to be an expert coder or have very competitive engineering experience to excel in this project. It's more about **correctly following the instructions to submit high-quality data!!**

Introduction to Terminal Coding Agent [Recommended Reading]

Unlike usual coding-related topics you'd chat with an LLM in your mobile app or a web interface, **coding agents** are AI tools that go beyond chatbot-style conversation and code running sandbox. At the minimum, they can perform simple autocomplete or suggestion, while they can also **reason about your project, plan changes, run commands/tools, modify files, commit changes**, etc. Typical capabilities of agentic coding tools include:

- They ingest/understand a **whole codebase context** (not just the current file).
- They can **plan** multi-step tasks: e.g., "Add feature X, create tests, update docs".
- They can **execute** commands in the dev environment: run tests, lint, commit code, open browser, search web, etc.
- They can **modify files** (create, edit, delete) and often integrate with version control.
- They may have **tooling support** (web search, terminal, file system, diffing) and sometimes a memory/context over the session.

Some common agentic coding tools available on the market include:

- [Cline](#) — an open-source TDD-focused coding agent (e.g. runs tests, fixes them until they pass), can be installed as extension in VS Code.
- [Cursor](#) — an AI-first IDE/editor (and sometimes CLI) tool focused on coding productivity.
- [Claude Code](#) / [Codex](#) — terminal/CLI agentic coding tools provided by LLM labs, deeply integrated with project context and tool-running capabilities.

Coding agents can be useful in a variety of scenarios such as:

- They help automate **boilerplate**, large refactors, multi-file changes.
- They reduce context switching (you stay in your editor/terminal rather than juggling many windows).
- They allow you to delegate a "developer-agent" a higher-level goal rather than micro-instructions.
- Useful especially for bigger tasks or unfamiliar codebases.



Setting up Coding Agents

In the first part of the qualification after the survey, you will follow the below instructions to set up GPT Codex, Gemini, or Claude Code and a terminal logging tool. There will be no need for an active subscription, you will perform authentication via a proxy. You can select either agentic CLI tool you feel more comfortable with.

- You are free to choose other CLI agents that you have access to, as long as you can get the transcript log properly!

⚠

You are being given access to models via a DataAnnotation proxy for this project. You are not permitted to use these models for any purpose that does not support completion of a task for this specific project, and all traffic through the proxy is logged and monitored. Abusing model access will be subject to penalties including but not limited to removal from the platform.

⚠ If you are on Windows, you will need to install [WSL](#) to proceed with this project, since `tmux` and most of the mentioned CLIs below are only supported on Mac and Linux.

▼ Tool `tmux` for Terminal Logging

To set up `tmux`, a tool that allows you to capture the conversation history in terminal to submit in this qualification, simply **follow the instructions below**.

▼ Installing `tmux`, increasing history limit, enabling mouse mode for scrolling

On MacOS:

You need to first install `homebrew` if not installed (or MacPorts): <https://brew.sh/> Click to copy command

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Add to `.zprofile` so the `brew` command is recognized (replace `USERNAME_HERE` with your username): Click to copy command

```
echo >> /Users/USERNAME_HERE/.zprofile
echo 'eval "$(/opt/homebrew/bin/brew shellenv)'" >> /Users/USERNAME_HERE/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Then, install `tmux` using homebrew: Click to copy command

```
brew install tmux
```

After installing tmux, run the following command in a terminal to enable mouse scrolling and increase the scrollback history limit:

Click to copy command

```
echo -e "set-option -g history-limit 100000\nset -g mouse on" > ~/.tmux.conf
```

This creates your `tmux` config file. `tmux` will automatically load this config the **next time you start a session**.

Already have `tmux` running?

If you already started `tmux` before running the command above, run this in a different terminal to apply the settings immediately to the `tmux` session you are working in: Click to copy command

```
echo -e "set-option -g history-limit 100000\nset -g mouse on" > ~/.tmux.conf && tmux source-file ~/.tmux.conf
```

On Linux:

Many distributions have prebuilt packages:

Platform	Install Command
----------	-----------------

Arch Linux	<code>pacman -S tmux</code>
Debian or Ubuntu	<code>apt install tmux</code>
Fedora	<code>dnf install tmux</code>
RHEL or CentOS	<code>yum install tmux</code>
openSUSE	<code>zypper install tmux</code>

After installing tmux, run the following command in a terminal to enable mouse scrolling and increase the scrollback history limit:

Click to copy command

```
echo -e "set-option -g history-limit 100000\nset -g mouse on" > ~/.tmux.conf
```

This creates your tmux config file. tmux will automatically load this config the **next time you start a session**.

Already have tmux running?

If you already started tmux before running the command above, run this in a different terminal to apply the settings immediately to the tmux session you are working in: Click to copy command

```
echo -e "set-option -g history-limit 100000\nset -g mouse on" > ~/.tmux.conf && tmux source-file ~/.tmux.conf
```

If yours is not listed there, try searching using your package manager, otherwise [refer to their github for instructions on compiling from source](#) (or switch distros).

Below are three terminal coding agent CLI tools that you can install. You can select just one to install, no need to install all three of them.

- If you experience too many technical difficulties with one tool, or you simply want experience with more of them, you can freely experiment with more than one. But your submission must be focused around just one tool.

OpenAI Codex (Linux, Mac, WSL)

Codex is OpenAI’s agentic coding companion available across terminal (Codex CLI), IDEs, and the web (Codex Web in ChatGPT), unified under your personal ChatGPT account. In this qualification, you will only work with the CLI version. Currently, Codex is only supported on Linux and Mac.

Part 1 — Installing Codex CLI

macOS or Linux (recommended via npm): Click to copy command

```
npm install -g @openai/codex
```

Verify install: Click to copy command

```
codex --version
```

[Refer to Codex CLI official docs for assistance if something went wrong.](#)

Part 2 — Sign in to Codex

After installing Codex, you need to test it with the proxy:

1. Add this line: `openai_base_url = "https://app-llmproxy.dataannotation.tech/api/llm_proxy/openai/v1"` to your `config.toml` for Codex versions above v0.120.0. This will allow you access to Codex via the proxy.
 - You must use version Codex v0.120.0 or above for this project, and this method must be used as the old `OPENAI_BASE_URL` is deprecated.

- Your `config.toml` file can be found in `~/.codex/config.toml` for user-level configuration, or `.codex/config.toml` for project-level configs. For help, [see the docs](#).

2. **Set the required environment variables** in your terminal: Click to copy 

```
export OPENAI_API_KEY=KEY_AVAILABLE_WHEN_TASK_STARTED
```

3. Run `codex` in the same terminal.
4. If prompted for your authorization method, select the option indicated via an API key. After doing so, it should automatically show the API key from the environment variable you set above. Press enter to confirm.
5. Exit Codex (`/quit` in Codex Click to copy )

To verify everything works, run this in the same terminal (not inside Codex): Click to copy 

```
mkdir ~/codex-test && cd ~/codex-test && codex 'What is in this folder?'
```

Codex should respond if setup was successful. If you are already logged in via a personal account, you might see "401 Unauthorized" errors. If this is the case:

- Run `/logout` in Codex to log out of your personal account.
- Make sure the `OPENAI_API_KEY` env var is set correctly in the terminal.
- Start Codex again and you'll see an option to "Provide your own API key". Select this option. The API key should be pre-filled, so you only need to press Enter.

Then exit and clean up:

- `/quit` in Codex Click to copy 
- Delete the test directory: `cd ~ && rm -rf ~/codex-test` Click to copy 

✓ **Claude Code (Linux, Mac, WSL)**

Claude Code is a terminal coding agent product built by Anthropic, and Claude Code is only supported on Linux and Mac. [Windows users can install WSL for support](#).

✓ **Part 1. Installing Claude Code**

1. **Install native Claude Code CLI**

```
curl -fsSL https://claude.ai/install.sh | bash
```

2. **Verify Installation** Click to copy 

```
claude --version
```

Refer to the [Claude Code Docs](#) for assistance if something went wrong.

✓ **Part 2. Signing in to Claude Code**

To authenticate with Claude Code, you will be using a proxy.

(Run these commands in a new terminal emulator, not in Claude Code)

1. After installing Claude Code, you need to test it with the proxy:
1. **Set the required environment variables** in your terminal: Click to copy 

```
export ANTHROPIC_API_KEY=KEY_AVAILABLE_WHEN_TASK_STARTED
export ANTHROPIC_BASE_URL=https://app-llmproxy.dataannotation.tech/api/llm_proxy/anthropic
```

2. Run `claude` in the same terminal and select a color scheme.
3. If prompted to use the detected API key, select "Yes".
 - If you accidentally selected "No", run `/config` in Claude Code and set `Use custom API key` to `true`.
4. Press `Enter` to accept the recommended default terminal settings, then `Esc` to exit.

To verify everything works, run this in the same terminal (not inside Claude Code): [Click to copy](#)

```
mkdir ~/claude-code-test && cd ~/claude-code-test && claude 'What is in this folder?'
```

Claude should respond if setup was successful. If you are already logged into a Claude Code account, you may see a warning about *"Auth conflict: Using ANTHROPIC_API_KEY instead of Anthropic Console key..."*. This is expected and means the proxy API key is being used correctly. You can ignore it, or run `/logout` to remove the warning.

Then exit and clean up:

- `/exit` in Claude Code [Click to copy](#)
- Delete the test directory: `cd ~ && rm -rf ~/claude-code-test` [Click to copy](#)

✓ Gemini CLI (Linux, Mac, Windows)

Note: Although Gemini CLI supports Windows, if you are using Windows you may still need to use WSL for tmux.

Gemini is Google's family of AI models that assist with tasks like coding and writing. In this project, you will only work with the CLI version. Gemini is supported on Linux, macOS, and Windows.

✓ Part 1: Installing Gemini CLI

macOS or Linux (recommended via npm): [Click to copy](#)

```
npm install -g @google/gemini-cli
```

Verify install: [Click to copy](#)

```
gemini --version
```

[Refer to Gemini CLI official docs for assistance if something went wrong.](#)

✓ Part 2: Setting up Google Gemini Authentication

For Gemini CLI, you will make use of a proxy for authentication.

1. **Set the required environment variables** (in the terminal where you intend to have the conversation): [Click to copy](#)

```
export GEMINI_API_KEY=KEY_AVAILABLE_WHEN_TASK_STARTED
export GOOGLE_GEMINI_BASE_URL=https://app-llmproxy.dataannotation.tech/api/llm_proxy/gemini
```

To test that the process worked, in the same terminal where you set the environment variables, type the following command:

[Click to copy command](#)

```
mkdir ~/gemini-test && cd ~/gemini-test && gemini -i "What is in this folder?"
```

If you see Gemini respond, you're good. Exit the TUI (`/quit`) and clean up: [Click to copy command](#)


```
cd ~ && rm -rf ~/gemini-test
```

Create a Conversation with the Agent

Now you've configured a terminal coding agent in your local machine, the next part of the qualification will require you to create a conversation with the agent on any of the below 2 topics and do some light evals. You'll need to configure `tmux` to export your conversation too!

- **Topic 1:** Submitting 1 **ambiguous prompt** with a codebase that results in model failures in correctly interpreting the ambiguity and go straight ahead with verbose and inaccurate assumptions.

▼ Types of Ambiguity

There are 6 types of ambiguity that you can incorporate in your prompts to the model. Please roughly follow the ambiguity categories distribution below for all of your tasks:

1. Underspecified Requirements

▼ Definitions & Examples

Underspecified Requirements: Requirements missing critical details that affect implementation

- "Add authentication to the app" (what type? OAuth, sessions, JWT? For which routes?)
- "Make the API faster" (which endpoints? What's the target?)
- "Improve error handling" (where? What specific scenarios? What's the current behavior?)
- "Add data validation" (which fields? What validation rules? Client-side, server-side, both?)

Good response

Makes a reasonable attempt at interpretation based on what's clear from the request (e.g., implementing basic session auth if "add authentication" clearly implies starting with something simple), while explicitly stating assumptions and asking specific clarifying questions about ambiguous aspects (OAuth providers? Password requirements? Session duration?). **The balance between attempting implementation versus asking questions first will vary based on context - trust your judgment as an annotator to prefer the response that handles the ambiguity most appropriately for the specific task.**

Bad response

Implements elaborate OAuth with Google/GitHub/Facebook without asking, or refuses to do anything until every detail is specified.

2. Technical Impossibilities

▼ Definitions & Examples

Technical Impossibilities: Requests that violate technical constraints

- "Make this Python 2.7 code use `async/await`" (`async/await` requires Python 3.5+)
- "Add this feature without modifying the database schema" (when schema change is necessary)
- "Make this $O(n^2)$ algorithm $O(\log n)$ " (when problem fundamentally requires $O(n^2)$)
- "Import this package" (when package doesn't exist or is incompatible)

Good response

Explains why the exact request is impossible, proposes closest alternative, asks if alternative is acceptable.

Bad response

Silently does something different, or implements a broken/incorrect solution to "fulfill" the request.

3. Contextual Ambiguity

▼ Definitions & Examples

Contextual Ambiguity: Ambiguity arising from incomplete codebase context

- "Fix the bug in the user service" (multiple bugs exist)
- "Use the existing email validation" (no such validation exists)
- "Follow the same pattern as the other endpoints" (endpoints use inconsistent patterns)
- "Update the config like we discussed" (no prior discussion in context)

Good response

Searches codebase, identifies the ambiguity (e.g., "I found 3 potential issues in user_ service.py"), asks which to address.

Bad response

Picks one arbitrarily without acknowledgment, or claims to have context that wasn't provided.

4. Interpretation Ambiguity

▼ Definitions & Examples

Interpretation Ambiguity: Multiple valid interpretations of the same request

- "Sort the results by relevance" (relevance can mean recency, popularity, text match score, etc.)
- "Add pagination" (cursor-based? offset-based? what page size? return total count?)
- "Cache the API responses" (in-memory? Redis? for how long? cache key strategy?)
- "Add logging" (what level? which operations? what format? where?)

Good response

Implements one reasonable interpretation, explains the choice, asks if a different approach is preferred.

Bad response

Implements without acknowledging other valid interpretations, or over-engineers trying to support all possibilities.

5. Conflicting or Contradictory Requirements

▼ Definitions & Examples

Conflicting or Contradictory Requirements: Constraints that conflict with each other

- "Add this feature but don't change any existing code" (feature requires modifications)
- "Make it secure but don't add any authentication" (contradictory goals)
- "Support IE11 and use modern ES2020 features" (incompatible)
- "Keep backward compatibility but change the API response format" (likely breaks compatibility)

Good response

Identifies the conflict, explains the tradeoffs, proposes a resolution, asks for prioritization.

Bad response

Silently prioritizes one constraint without acknowledging the conflict.

6. False Ambiguity (Clear from Context)

▼ Definitions & Examples

False Ambiguity (Clear from Context): Requirements that appear underspecified but are actually clear when examining README, comments, existing code, documentation, config, or previous conversation

- "Fix the validation bug" (when README or comments specify exact validation rules that aren't being followed)
- "Make the endpoint return the correct status codes" (when API documentation clearly defines which status codes to use)
- "Implement the missing error case" (when code comments have TODO noting specific error condition to handle)
- "Add the required field to the response" (when API spec in README lists all required response fields)
- "Handle the edge case" (when inline comments explicitly describe the edge case that needs handling)
- "Make it work with the database schema" (when schema is defined in migration files or comments)

Good response

Examines README, comments, and existing code to understand conventions, implements consistently and correctly with documented/evident patterns, briefly confirms "Based on the README section on error handling, I'm using..." or "Following the pattern in user [_controller.py](#) where..." without over-questioning.

Bad response

Either (1) asks unnecessary clarifying questions about things clearly documented in README or evident from consistent codebase, failing to check available context, or (2) ignores relevant documentation and code that contains useful information, implementing something wrong or inconsistent without examining the codebase.

- **Topic 2:** Submitting 1 codebase deep research prompt with a codebase that result in inaccurate text explanation from the agent.

▼ Question Categories and Examples

There are 10 different question categories that you can incorporate in your prompts to the model. Please roughly follow the distribution below for all of your tasks:

1. Execution Flow Tracing

▼ Definitions & Examples

Execution Flow Tracing: Questions requiring understanding of how data/control flows through the system:

- "How does a user request get processed from the HTTP endpoint to the database query?"
- "What happens when the cache is invalidated? Trace the full cascade of events."
- "Walk me through what occurs when a webhook is received, from entry point to storage."
- "How does error handling propagate from the service layer back to the client?"
- "What's the complete lifecycle of a background job from creation to completion?"

Good response

Traces the flow step-by-step with file references: "Request enters at [api/routes.py:45](#) → calls [service/user_service.py:123](#) → queries via [db/repositories/user_repo.py:89](#) → returns through middleware at [middleware/response.py:34](#)"

Bad response

It processes the request through the API layer and saves to the database (no specifics, no file references)

2. Architecture & Design Patterns

▼ Definitions & Examples

Architecture & Design Patterns: Questions about structural decisions and patterns:

- "What design patterns are used for dependency injection in this codebase?"
- "How is the separation between business logic and data access maintained?"
- "What's the plugin/extension architecture? How would I add a new plugin?"
- "How does this codebase handle configuration across environments?"
- "What's the strategy for managing database migrations?"

Good response

The codebase uses a factory pattern in [core/factory.py](#) where dependencies are registered (lines 12-45). Services receive dependencies via constructor injection (see [service/payment_service.py:10](#)). The `@inject` decorator in [utils/di.py:67](#) handles resolution.

Bad response

It uses dependency injection (no evidence, no explanation of how)

3. State Management & Side Effects

▼ Definitions & Examples

State Management & Side Effects: Questions about how state is maintained and modified:

- "Where is session state stored and how is it shared across requests?"
- "What are all the places where the User model gets updated?"
- "How does this application handle database transactions and rollbacks?"
- "What side effects occur when deleting an account? (cascade deletes, cleanup, notifications, etc.)"
- "How is cache consistency maintained when underlying data changes?"

Good response

Identifies all state mutation points: "User model updates occur in: (1) [auth/register.py:78](#) during signup, (2) [profile/update.py:45](#) for profile changes, (3) [admin/users.py:123](#) for admin modifications, (4) [background/sync.py:234](#) during external syncs"

Bad response

Lists one or two obvious places, misses important ones

4. Error Handling & Edge Cases

▼ Definitions & Examples

Error Handling & Edge Cases: Questions about robustness and failure modes:

- "What happens if the external API call times out? How is it handled?"
- "How does the system handle race conditions when two users modify the same resource?"
- "What validation exists to prevent invalid data from reaching the database?"
- "How are partial failures handled in batch operations?"
- "What happens if the background worker crashes mid-job?"

Good response

Traces error paths: "Timeouts are caught in [api/client.py:89](#) with a 30s limit. The exception triggers retry logic in [utils/retry.py:23](#) (3 attempts with exponential backoff). After exhaustion, it's logged via [logging/error_handler.py:45](#) and a fallback response is returned in [api/client.py:102](#)."

Bad response
"There's error handling" or describes hypothetical handling that doesn't exist in the code

5. Dependencies & Coupling

▼ Definitions & Examples

Dependencies & Coupling: Questions about relationships between components:

- *"What are all the dependencies of the UserService? Direct and transitive?"*
- *"If I wanted to swap out the email provider, what files would need changes?"*
- *"Which modules depend on the legacy authentication system?"*
- *"What's the data flow between the frontend and backend for this feature?"*
- *"How tightly coupled is the payment processing to the rest of the system?"*

Good response
Maps dependency graph with specifics: "UserService ([service/user.py](#)) directly depends on: EmailService (injected at line 12), UserRepository (line 13), CacheManager (line 14). Transitive dependencies include: SMTP client via EmailService, Redis via CacheManager, PostgreSQL via Repository base class."

Bad response
"It depends on several services" (no enumeration, no analysis)

6. Performance & Optimization

▼ Definitions & Examples

Performance & Optimization: Questions about efficiency and resource usage:

- *"What are the potential performance bottlenecks in the search functionality?"*
- *"How does the application handle pagination for large datasets?"*
- *"Where are database queries potentially causing N+1 problems?"*
- *"What caching strategies are employed and where?"*
- *"How is memory managed for large file uploads?"*

Good response
Identifies issues with evidence: "N+1 query in [api/posts.py:67](#) - loops over posts and calls `user_repo.get(post.user_id)` for each. Should use eager loading with join. Confirmed by query logs showing 100+ individual SELECT queries for [/api/posts](#) endpoint."

Bad response
"The search could be faster" (no specific analysis)

7. Security & Authorization

▼ Definitions & Examples

Security & Authorization: Questions about access control and security mechanisms:

- *"How is authorization enforced for API endpoints?"*
- *"What prevents users from accessing other users' data?"*
- *"How are secrets and credentials managed?"*
- *"What SQL injection protections exist?"*
- *"How is input sanitization handled before rendering to users?"*

Good response
Traces security checks: "Authorization uses decorator pattern. `@require_auth` in [auth/decorators.py:23](#) validates JWT. `@require_permission('admin')` in [auth/decorators.py:45](#) checks role. Applied to admin routes in [api/admin_routes.py](#). Row-level security in [db/policies.py:12](#) ensures queries filter by `user_id=current_user.id`."

Bad response
"There's authentication on the routes" (no details on how it works)

8. Testing & Observability

▼ Definitions & Examples

Testing & Observability: Questions about testing coverage and monitoring:

- *"What aspects of the payment flow are covered by tests?"*
- *"How does the logging infrastructure work? What gets logged where?"*

- *"What metrics are collected and where are they sent?"*
- *"How would you test the webhook processing without hitting external services?"*
- *"What monitoring exists for background job failures?"*

Good response

Analyzes test coverage: "Payment flow tests in [tests/test_payments.py](#) cover: (1) successful charge (line 45), (2) declined card (line 78), (3) network timeout (line 102). Missing: refund flow, webhook validation, idempotency checks. Mocking via [tests/fixtures/stripe_mock.py:23](#)."

Bad response

"There are tests for payments" (no coverage analysis)

9. Data Models & Schema

▼ Definitions & Examples

Data Models & Schema: Questions about data structure and relationships:

- *"What's the complete schema for the User model, including all relationships?"*
- *"How is the many-to-many relationship between Projects and Users implemented?"*
- *"What constraints exist on the database schema level?"*
- *"How is data versioning/history tracking implemented?"*
- *"What indexes exist and what queries do they optimize?"*

Good response

Complete schema analysis: "User model ([models/user.py:12](#)) has fields: id (UUID, PK), email (unique index), created_at, updated_at. Relationships: [posts](#) (one-to-many via [Post.user_id](#) FK), [teams](#) (many-to-many via [team_members](#) junction table in [models/team.py:34](#)). Unique constraint on (email, tenant_id) in migration [migrations/0023_unique_email_per_tenant.py](#)."

Bad response

"User has fields like email and relationships to posts" (incomplete, no specific references)

10. Legacy Code & Technical Debt

▼ Definitions & Examples

Legacy Code & Technical Debt: Questions about old code and migration paths:

- *"What's the difference between the old and new authentication systems?"*
- *"Which parts of the codebase still use the deprecated XML API?"*
- *"How does the legacy database schema differ from the new one?"*
- *"What's the migration plan for moving from sync to async handlers?"*
- *"Where are TODO comments indicating incomplete refactoring?"*

Good response

Identifies technical debt: "Two auth systems coexist: (1) Legacy session-based in [auth/legacy.py](#) used by [/old_api/](#) routes, (2) JWT-based in [auth/jwt.py](#) used by [/api/v2/](#). Legacy system has TODO at line 234: 'Remove after Q2 2024 migration'. 15 endpoints still on old system (found via grep for [@legacy_auth](#))."

Bad response

"There's some old code" (no specifics, no analysis)

Both of the allowed prompt topics require a codebase context for the conversation. Please see the below for codebase requirements. You don't need to follow exactly since this is just a practice project for qualification!

▼ Codebase Requirements - Minimal But Realistic Codebase

- **Small projects:** 3-50 files, 200-10000 lines total (roughly)
 - You can use bigger projects if that's easier for you.
- **Clear structure:** Standard project layout (src/, tests/, etc.)
- **Intentional gaps:** Missing error handling, incomplete features, TODO comments
- **Subtle inconsistencies:** Different patterns in different files (to create interpretation ambiguity)
- **Documentation:** README with partial info, some docstrings
- **File Size:** Try to keep the total file size of the initial codebase to less than ~1GB. This is the current limit on file uploads for this project.

- It is fine if you go over the limit, however, you will likely need to remove any extra dependencies such as node modules to successfully be able to upload the tarball for the codebase.

Note that you do **NOT** need to proceed with your conversation with follow-up prompts (assuming your first prompt successfully challenges the model already.) Submitting **single-turn conversations are more than enough** for this qualification!

✓ **Configure tmux and start your conversation with the agent.**

✓ **Part 1. Cloning repo, checkout reference commit, starting the agent in tmux, and entering the starting prompt**

Firstly, you will need to set up the repo for your conversation.

If using a publicly-available GitHub repo:

1. Clone the repository: `git clone <YOUR_REPO_URL>`
2. Enter the repository directory: `cd <REPO_FOLDER_NAME>`
3. Check out the specific commit you want to use (if you have a specific commit hash): `git checkout <COMMIT_HASH>`
4. Go back to the parent directory: `cd ../`
5. From the parent directory, create a tarball (replace `<YOUR_REPO_FOLDER>` with the actual repo directory name): `tar -cf ./<REPO_FOLDER_NAME>-start.tar ./<REPO_FOLDER_NAME>`

If using a local repo:

1. Navigate to the parent directory containing your repo and make a copy of the repo: `cp -r <REPO_FOLDER_NAME> ~/claude-project/<REPO_FOLDER_NAME>`
 - Working on the copy is important so that you don't accidentally modify your original code
 - **IMPORTANT:** Your repo folder name should use only lowercase letters, numbers, and hyphens (e.g., `my-project-name`). Rename the folder if needed before proceeding.
2. Navigate to the parent directory of the copy you just made: `cd ~/claude-project/`
3. From the parent directory, create a tarball (replace `<YOUR_REPO_FOLDER>` with the actual repo directory name): `tar -cf ./<REPO_FOLDER_NAME>-start.tar ./<REPO_FOLDER_NAME>`

Then, follow these steps:

1. **Set the required environment variables for your chosen CLI tool** in the terminal where you have the project open, following the steps above ([Claude Code](#), [Codex](#), [Gemini](#)).
2. ⚠️ **Kill any existing tmux sessions:** `tmux kill-server` [Click to copy](#)

- Tmux sessions only inherit environment variables set before the session was created. Reattaching to a pre-existing session means your proxy variables won't be available. **Killing the server ensures a fresh start.**
 - This kills all tmux sessions. Make sure you don't have other sessions with unsaved work.
3. **Start a tmux session** in the same terminal where you set the environment variables: `tmux` [Click to copy](#)
4. **Launch your chose CLI:**
 - **Claude Code:** `ANTHROPIC_MODEL=claude-opus-4-6 CLAUDE_CODE_EFFORT_LEVEL=medium claude` [Click to copy](#)
 - **Codex:** `codex --sandbox workspace-write --ask-for-approval on-request --model gpt-5.4 --config model_reasoning_effort="medium"` [Click to copy](#)
 - **Gemini:** `gemini --model gemini-3.1-pro-preview` [Click to copy](#)
5. **Send your opening prompt and work with the model.**
6. **Interact with the agent**
 - Write your starting prompt under any of the above 2 topics to begin the conversation.
 - Check if the response is bad. If your response looks good, you can either send a second prompt to ask a similar question, or exit the conversation and restart the agent and `tmux`.
7. **Export the tmux transcript** (see "How to Capture Your tmux Transcript" below) and upload it to the corresponding question item.
 - **IMPORTANT:** Export the tmux transcript before exiting tmux.
8. **Exit the CLI Tool:** `/exit` or `/quit`.

▼ **Part 2. Exporting & Uploading the Conversation Transcript**

Export the transcript before closing tmux.

When you are done with a model session, do the following:

- Press `Ctrl+b` , then `:` to open tmux command mode (the green bar at the bottom turns yellow). Enter the save command for your model:
`capture-pane -S -; save-buffer ~/<REPO FOLDER NAME>-log.log; delete-buffer`

If the green bar does not turn yellow, you might not have pressed the key sequence correctly.

- Hold `Ctrl` and press `b` , release both, then hold `Shift` and press `;` (to type `:`)
- You can check your tmux prefix key with `tmux show -g prefix`

After you have successfully uploaded the log file, and filled in the relevant question fields, you can submit the task! Please note that if you pass the qual, it may take a few business days for relevant projects to appear on your dashboard.

Thank you for taking this qualification :)

➤ **Frequently Asked Questions / Troubleshooting - Please Review before asking a question in the chat!**

Chat with workers

➤ Send



Last draft saved at 7:18 PM
[Clear Response](#)

Please only leave messages if you have questions about performing the task that you need the poster to answer, if the project is broken in some way, or if you have tips or answers for other workers. If you are facing technical difficulties, have feedback about pay, or have other suggestions, please email DataAnnotation instead. Thanks!

▼ 📌 Pinned messages (10)

Admin AR: You can use the /export command in Claude Code to get the transcript if your tmux capture fails for some reason.

Admin AR: Hey everyone! We've updated the instructions to put the tmux capture step before the CLI exit step. This is due to multiple reports of issues with tmux capture recently. If you were impacted by this and ended up uploading a log obtained by other means (e.g., using the /export option), that is fine. Thanks for your patience and apologies for the inconvenience!

Admin AR: Hey everyone! Before asking questions in the chat, please read the FAQ section in the instructions and the pinned chat messages. If that doesn't resolve your concern, please use the browser's search option to look for keywords that might match your concern and and quickly scan to see if your request has already been addressed below. Thanks!

Admin AR: Also, some of you may see a background check project on your dashboards. If you do, please complete it to gain access to some of the projects that this qualification unlocks. Thanks!

Admin AR: Hey everyone! If you exit to dashboard after your first attempt, that is fine! You do not have to take the second attempt necessarily. Even if you want to, you don't have to do it right away. You can come back at any time as long as the qualification is active!

Admin AR: Hey everyone! Not submitting a second attempt does NOT block you from qualifying. You will see relevant project eventually if you pass. It can take a few business days / weeks for relevant projects to show up on your dash.

Admin AR: Hey everyone! Before asking questions in the chat, please refer to the FAQ section of the instructions to see if your questions have already been answered there. Thanks!

Admin AR: If you saw your submission under the Report Time section and reported your time, that means your submission went through. You still see this qualification because you are given 2 attempts, as noted in the instructions. You are not required to attempt a second time but you can if you are not confident in your first submission!

Admin AR: For everyone facing the "401 Unauthorized" issue, please try to log out of your personal accounts using `/logout` in codex. Then, when you start codex up again, you'll see an option to "Provide your own API key" if you've set the environment variable correctly. Select that option (the API key will be pre-filled; just press Enter) and you should stop seeing the error!

Admin CP: For those that are experiencing issues using the proxy auth in a particular CLI tool (such as Codex) despite being sure you've performed the setup correctly, feel free to try out a different CLI Tool in the meantime to see whether those tools will work for you!

gemini cli is taking forever to reply to "What is in this folder?"
Its been 4+ minutes. Earlier attempts also had the same issue with "[API Error: An unknown error occurred.]" message after 5 mins or so

ChinmayB-43VY3XZJ

4 hours ago

They mentioned Cursor for agentic workflows in here. Does this specifically have to be terminal output or can i just use cursor and provide the Cursor chat history with an appropriate model selected?

DamanveerS-F6G32MRQ

yesterday

↩ Replying to @MaciejN-XJ29HA3F: As a tip to other people, I dont think its even possible to induce a failure in Topic 2 if you just put in one question in the prompt with ...

hi, a quick question. Once i see that the model provided bad response to the opening prompt and attempts to modify the code automatically, do i need to let it proceed with modifying the code as it wants to, or is seeing that wrong initial response is enough, and i just export the chat once the claude code hits the prompts of asking me to press the button to let the model do the code changes?

RomanH-AWZME7HG

yesterday

Hello, I submitted this qualification a couple days ago and the log file from tmux. I wanted to add that I have also used Gemini AI to do extensive research on a variety of subjects of interest as well as compiling product choices and doing comparisons and choosing based on availability in different cities. Is it possible to add this to my answers? And I just wanted to confirm if my submission completed successfully. The qualification indicates one task is left but it looks like it's the same task I already completed. Your guidance is helpful!

MichaelIM-FCAQRVE9

Friday

I get API error when trying to talk to Claude. It worked last night - today not -- what can that be?

JohnC-F23VHMW9

Friday

my tmux log does not show my full interaction with claude code

ChinmayB-43VY3XZJ

Friday

I have claude already and i have been unable to get Claude to run with the link provided and the API key. i am about out of time. what do i do?

ScottB-VZEQWRYT

Friday

↩ Replying to @DavidF-D64GY3QD: I'm finished with the main body of work, but cannot get my .tar file to upload. After several attempts, it is still hung on 0%. I'd like to...

☆ Approved by admin

To others who see this: This is not actually a bug. The download takes quite some time, and does not give much feedback (jumping as much as 33% after a very long pause where nothing seems to be happening). Simply waiting for much longer than I expected fixed the issue.